

1. Título y Autor

Título de la Propuesta: Implementación del Ecosistema Distribuido Medipass mediante Arquitectura Hexagonal para el Agendamiento Crítico de Especialidades y Sincronización Asíncrona de Historias Clínicas.

Autores:

Santiago Alzate Diego Alejandro Rodriguez Sergio Alvarez Hernandez
Mateo Cespedes Yehicol Andrés Hincapié.

Fecha: 29/05/2026

2. Resumen

Esta propuesta detalla la evolución y diseño arquitectónico del ecosistema **Medipass**, una plataforma distribuida de alta complejidad diseñada para la gestión crítica de citas médicas especializadas. El sistema mitiga el error humano y evita problemas operativos graves como el *Double-Booking* mediante la separación de responsabilidades en tres microservicios especializados, desarrollados bajo el patrón de **Arquitectura Hexagonal** y orquestados mediante **Docker Compose**.

- **MS-AgendaHub (Microservicio Principal):** Centraliza el núcleo del dominio del agendamiento. Utiliza **PostgreSQL** para la persistencia de datos relacionales y expone una API REST documentada con Swagger. Aplica de forma estricta las reglas de negocio para el bloqueo de agendas en tiempo real.
- **MS-Insurance (Microservicio Síncrono):** Actúa como un proxy/adaptador especializado para la comunicación síncrona vía HTTP con las aseguradoras externas, dictaminando de manera inmediata si un procedimiento está cubierto o no. Su persistencia está basada en **Oracle**.
- **MS-EHRLogger (Microservicio Asíncrono):** Registra de forma inmutable y asíncrona los resúmenes de las citas confirmadas en el Historial Clínico Digital (EHR). Consume eventos desde un broker de mensajería (**RabbitMQ**) y persiste la información en **MongoDB** para asegurar una escritura de logs rápida, elástica y de esquema flexible

Toda la infraestructura incluye un punto de entrada único gestionado por un **API Gateway** y un stack de observabilidad de última generación (**Prometheus, Grafana**) para garantizar la trazabilidad distribuida de los flujos.

3. Motivación

En entornos de salud de alta complejidad, el agendamiento manual o centralizado ineficiente genera pérdidas económicas masivas y pone en riesgo la vida de los pacientes por demoras en la atención. **Medipass** resuelve esto automatizando tres validaciones críticas: la verificación en tiempo real de cobertura del seguro, la eliminación absoluta del doble agendamiento (**Double-Booking**) y la actualización transparente del historial clínico sin degradar el tiempo de respuesta telefónico o digital hacia el usuario.

Justificación técnica

- **Desacoplamiento y Alta Cohesión:** Unificar la lógica de pólizas de seguros, agendas médicas e historias clínicas en un solo monolito crearía un sistema frágil y propenso a fallas en cascada. Al aislar **MS-AgendaHub**, protegemos la disponibilidad del núcleo del negocio.
- **Rendimiento en Escritura (EHR):** La actualización del historial clínico digital genera grandes volúmenes de datos históricos estructurados de forma variable. Delegar esto a **MS-EHRLogger** operando con **MongoDB** evita la contención de recursos y bloqueos en la base de datos relacional de la agenda principal.
- **Consistencia Eventual:** La agenda del médico no necesita esperar una respuesta confirmada del sistema de historias clínicas para liberar la llamada del operador; la mensajería asíncrona asegura que el dato llegará de forma confiable sin penalizar la experiencia de usuario (UX).

4. Diseño Técnico

La solución implementa una **Arquitectura Hexagonal** en cada microservicio para independizar las reglas de negocio de los detalles tecnológicos (frameworks, bases de datos, transportes).

Patrones de Diseño Implementados

1. **Adapter Pattern:** Utilizado en la capa de infraestructura para transformar las peticiones HTTP externas (como las de la aseguradora) y las interfaces de las bases de datos en contratos entendibles por el dominio.
2. **Strategy Pattern:** Implementado en el núcleo de validación de seguros para aplicar dinámicamente diferentes algoritmos de cobro o reglas de cobertura según la aseguradora del paciente.
3. **Factory / Builder Pattern:** Utilizado para la creación limpia e inmutable de las entidades complejas del dominio (**Cita, Paciente, HistorialLog**) garantizando que ningún objeto de negocio se instancie en un estado inválido.

A. MS-AgendaHub (Microservicio Principal)

- **Integración:** Coordina el flujo. Guarda una referencia externa (UUID) del paciente y del médico.
- **Capa de Dominio:** Contiene las entidades base (Cita, Medico, Horario), las validaciones programáticas y el servicio de dominio que ejecuta la lógica para impedir el *Double-Booking*. Totalmente libre de dependencias externas.
- **Capa de Aplicación:** Define los puertos de entrada (Casos de uso como AgendarCitaUseCase) y puertos de salida (ICitaRepositoryPort, IInsuranceServicePort, IEHRMessagePort).
- **Capa de Infraestructura:** Implementa adaptadores para la base de datos **PostgreSQL**, controladores REST documentados con **Swagger/OpenAPI** y los clientes HTTP para conectarse de manera síncrona con el servicio de seguros.

B. MS-Insurance (Microservicio Síncrono)

- **Integración:** Responde a peticiones HTTP directas originadas por el *MS-AgendaHub*.
- **Función:** Consulta las estructuras de pólizas mapeadas en una base de datos **Oracle**. Si la aseguradora devuelve un código de *"Procedimiento No Cubierto"*, el caso de uso del dominio frena inmediatamente la confirmación de la cita.

C. MS-EHRLogger (Microservicio Asíncrono)

- **Integración:** Escucha de manera pasiva el broker de mensajería. Cuando MS-AgendaHub publica el evento CitaConfirmadaEvent, este servicio lo consume en segundo plano.
- **Función:** Procesa el resumen de la cita y lo inserta de manera inmutable en **MongoDB**, asegurando que el historial clínico digital se mantenga actualizado de forma asíncrona.

Nota: Los 2 diagramas están en un mismo archivo de draw.io, cada uno en una pagina independiente

DIAGRAMAS C4:

<https://drive.google.com/file/d/1bnwlki5hsP8sAqMCdCeY7UQVfKw4t9Gz/view?usp=sharing>

5. Alternativas Consideradas

Alternativa 1: Orquestación Síncrona Total (HTTP) para el Historial Clínico

- **Evaluación:** Que MS-AgendaHub llamara vía REST a MS-EHRLogger y esperara a que este guardara en base de datos antes de responderle al usuario.

- **Por qué se descartó:** Si el sistema de historias clínicas sufría una caída o degradación de rendimiento, bloquearía por completo el agendamiento de citas, aumentando el abandono de llamadas y violando el requerimiento de negocio de "no demorar la llamada de la cita".

Alternativa 2: Base de Datos Única Compartida (Shared Database)

- **Evaluación:** Centralizar las tablas de agenda, pólizas e historias clínicas en una única instancia de base de datos relacional.
- **Por qué se descartó:** Rompe el principio fundamental de autonomía de microservicios. Generaría un acoplamiento estructural severo donde un cambio de esquema para la aseguradora podría corromper el flujo de asignación de citas en emergencias, además de causar bloqueos de tablas (deadlocks) por la concurrencia de accesos.

6. Impacto

Disponibilidad y Resiliencia: El uso de colas de mensajería garantiza que si MS-EHRLogger no está disponible temporalmente, los eventos de historial clínico se acumulen de forma segura en el broker y se procesen una vez se restablezca el servicio, logrando consistencia eventual sin pérdida de datos.

Observabilidad: Prometheus y Grafana exponen y visualizan métricas críticas de salud (tiempo de respuesta HTTP, tasa de errores, consumo de CPU/Memoria).

Operación y Despliegue: Todo el ecosistema (los 3 microservicios, Nginx, PostgreSQL, Oracle, MongoDB, RabbitMQ/Kafka, Prometheus, Grafana y Jaeger) se levanta de forma determinista mediante **Docker Compose** con un único comando, facilitando entornos de desarrollo idénticos a producción.

7. Enlaces Relacionados

Repositorio GitHub StockPro: <https://github.com/Alzatiyo/Medipass.git>